UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM AND NETWORK PROGRAMMING

CEF488

LAB 2: Named Pipes, Advanced Multiplexing & Real-time Signals

Group 7

NAME	MATRICULE	SIGNATURE
EFUETNGONG GENESIS TAZISONG	FE23A041	
NEIL AMBAYIH	FE23A105	
TAKU YVETTE WUTOH	FE23A152	
TAMANTENG ABERLINE NGUEMUTANG	FE23A154	

Course Instructor: **Mr. FORCHA GLEN**

HONESTY REPORT

TASK	RESPONSIBLE MEMBER(S)	CONTRIBUTOR(S)	REVIEWER(S)
Prelab Questions			
Implementation			
Testing and debuging			
Report writing			

Contents

1. Introduction

This laboratory exercise focused on the design and implementation of a scalable logging system using Linux Inter-Process Communication (IPC) mechanisms. The project required the development of a centralized logging server capable of receiving and processing structured log messages from multiple independent client processes through the use of Named Pipes (FIFOs).

To efficiently monitor multiple communication channels simultaneously, the server employed the Linux epoll I/O event notification facility in edge-triggered mode. The system also explored the use of real-time signals as an alternative asynchronous notification mechanism.

The implementation demonstrates several advanced operating system concepts including:

- Named Pipes (FIFOs)
- Non-blocking I/O
- Event-driven programming
- epoll multiplexing
- Real-time signals
- Dynamic file descriptor management
- Atomic communication guarantees
- Resource cleanup and fault tolerance

2. Objective

The objective of this lab was to implement a scalable logging system where multiple independent processes send structured log entries to a central server using named pipes (FIFOs). The server uses epoll to monitor multiple FIFOs simultaneously and optionally explores real-time signals for asynchronous notifications.

The system was designed to:

- Support multiple concurrent clients
- Prevent blocking operations
- Ensure efficient event handling
- Preserve message integrity
- Allow runtime expansion without restarting the server
- Demonstrate advanced Linux systems programming concepts

3. Real-Life Scenario

The implemented architecture models a centralized log collector commonly used in enterprise systems and microservice-based infrastructures.

In modern distributed environments, several independent applications or services generate logs simultaneously. Rather than writing directly to separate files, each service can transmit structured log messages to a centralized logging server through dedicated FIFOs.

The logging server then:

- Collects log messages
- Filters or categorizes entries
- Adds timestamps and metadata
- Forwards critical events to monitoring systems
- Stores logs for debugging and auditing purposes

4. Key Concepts

Named Pipes (FIFOs)

Named pipes, also known as FIFOs, are special files used for communication between unrelated processes.

Unlike unnamed pipes created with pipe(), FIFOs:

- Exist as files within the filesystem
- Persist until explicitly removed
- Allow communication between unrelated processes
- Are created using mkfifo()
- Are accessed using standard file operations such as open(), read(), and write()

FIFOs provide a simple and efficient mechanism for inter-process communication in Linux environments.

Atomic Writes and PIPE_BUF

POSIX guarantees that any write operation to a pipe or FIFO whose size is less than or equal to PIPE_BUF will be atomic.

On Linux systems, PIPE_BUF is typically 4096 bytes.

This means:

- Messages smaller than PIPE_BUF cannot interleave with writes from other processes
- Each log entry remains intact
- Structured logging becomes reliable even with multiple concurrent writers

To preserve atomicity, each client ensures that every log message remains within the PIPE_BUF limit.

I/O Multiplexing with epoll

The server uses epoll for efficient monitoring of multiple file descriptors.

Compared to older mechanisms such as select() and poll(), epoll provides:

- Better scalability
- Lower CPU overhead
- Improved performance with large descriptor sets
- Efficient event-driven execution

The server operates in edge-triggered mode using EPOLLET.

Edge-Triggered Mode

In edge-triggered mode:

- Notifications occur only when new data arrives
- The application must read until read() returns EAGAIN
- System wake-ups are reduced
- Higher efficiency is achieved

Real-Time Signals

Real-time signals extend the traditional Linux signal system. Signals in the SIGRTMIN to SIGRTMAX range provide:

- Signal queuing
- Delivery ordering
- The ability to transmit additional data using sigqueue()

These signals can be used to notify a server that a specific FIFO has received new data.

Dynamic File Descriptor Management

The server was designed to support runtime addition and removal of FIFOs.

This capability allows:

- System scalability
- Runtime reconfiguration
- Addition of new clients without restarting the server

5. Theoretical Background

Named Pipes vs. Unnamed Pipes

Unnamed pipes are created using pipe() and exist only while processes hold references to them.

Characteristics of unnamed pipes:

- Typically used between parent and child processes

- Exist only in memory
- Disappear automatically after closure
- Require related processes

Named pipes differ because:

- They exist as filesystem objects
- Unrelated processes can access them
- They persist until removed with unlink()
- They are more suitable for server-client communication

Atomicity and PIPE_BUF

The PIPE_BUF limit determines the maximum size of an atomic write to a pipe or FIFO.

If a process writes data less than or equal to PIPE_BUF:

- The kernel guarantees uninterrupted delivery
- Data from multiple writers will not mix
- Message boundaries remain intact

However, if the write exceeds PIPE_BUF:

- Interleaving may occur
- Partial writes become possible
- Structured messages may become corrupted

Therefore, all client messages in this implementation were intentionally designed to remain within the PIPE_BUF boundary.

6. System Architecture and Design

Overall Architecture

The system consists of:

- A centralized logging server
- Multiple client processes
- Multiple FIFOs
- A control FIFO for runtime management

The server continuously monitors all registered FIFOs using epoll.

Each client sends structured log messages to a specific FIFO.

The server receives, parses, timestamps, and processes these log entries efficiently.

FIFO Structure

The primary FIFOs created during initialization include:

- /tmp/fifo1
- /tmp/fifo2
- /tmp/fifo3

A dedicated control FIFO was also created:

- /tmp/control

The control FIFO is responsible for runtime commands such as:

ADD /path/to/new_fifo

Non-Blocking Operation

All FIFOs were opened using the O_NONBLOCK flag.

This was necessary because:

- Blocking open() calls could suspend the server
- Blocking reads would reduce concurrency
- Non-blocking mode enables continuous event-driven execution

When operating in non-blocking mode:

- read() may return EAGAIN if no data is available
- The application must handle this condition correctly

Event Processing Workflow

When epoll_wait() reports an event, the server performs the following sequence:

- Detect activity on a FIFO
- Read incoming data
- Continue reading until EAGAIN is returned
- Reconstruct complete messages using newline delimiters
- Parse the log level
- Append timestamps
- Display or store formatted log entries

This workflow ensures efficient handling of multiple concurrent clients.

7. Implementation Tasks

Task 1: Core Logging Server

The first implementation stage involved building the centralized logging server.

Server Responsibilities

The server was responsible for:

- Creating FIFOs using mkfifo()
- Opening FIFOs in non-blocking mode
- Creating an epoll instance
- Registering descriptors with epoll_ctl()
- Processing incoming events
- Handling EOF conditions
- Managing resource cleanup

Event Handling Logic

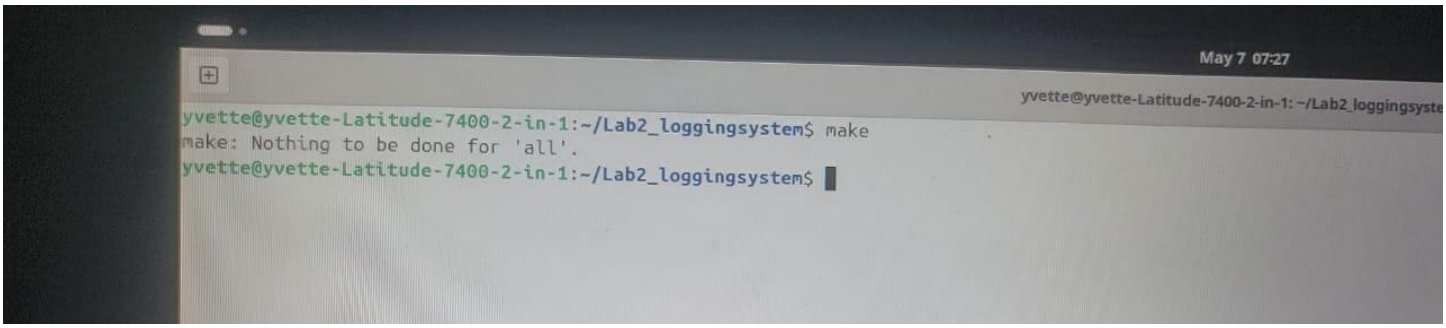
The event loop continuously calls epoll_wait() to monitor incoming events.

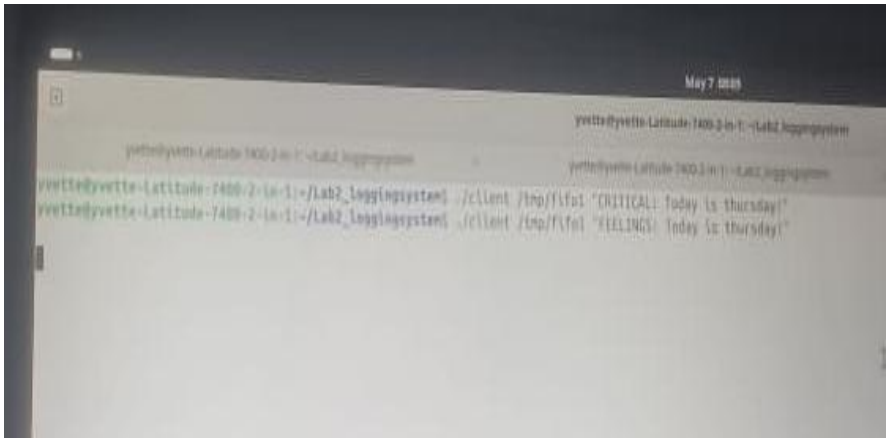
When activity is detected:

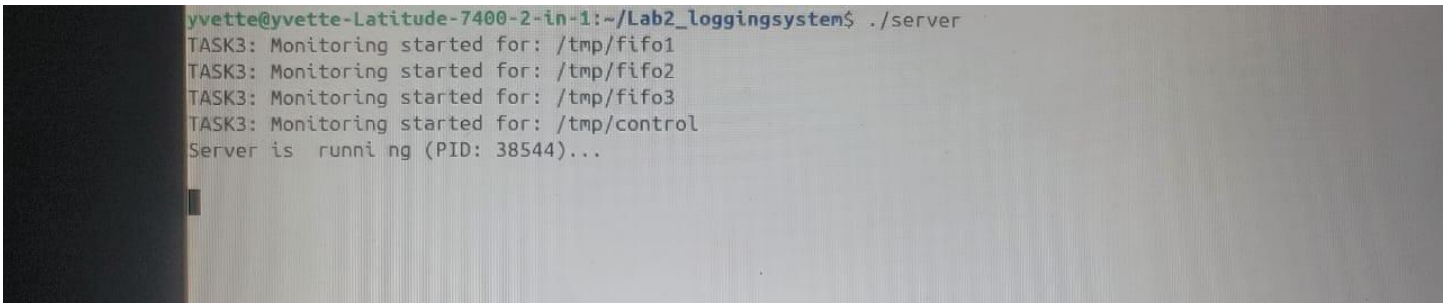
- Data is read from the corresponding FIFO
- Messages are reconstructed
- Severity levels are parsed
- Timestamps are generated

The server supports multiple message levels including:

- INFO
- WARNING
- CRITICAL


May 7 07:27
yvette@yvette-Latitude-7400-2-in-1: ~/Lab2_loggingsyste
yvette@yvette-Latitude-7400-2-in-1:~/Lab2_loggingsystem$ make
make: Nothing to be done for 'all'.
yvette@yvette-Latitude-7400-2-in-1:~/Lab2_loggingsystem$




yvette@yvette-Latitude-7400-2-in-1:~/Lab2_loggingsystem$ ./server
TASK3: Monitoring started for: /tmp/fifo1
TASK3: Monitoring started for: /tmp/fifo2
TASK3: Monitoring started for: /tmp/fifo3
TASK3: Monitoring started for: /tmp/control
Server is runni ng (PID: 38544)...

Task 2: Client Application

The client application acts as a log producer.

Client Responsibilities

Each client:

- Opens a designated FIFO
- Sends a formatted log message
- Closes the FIFO after transmission

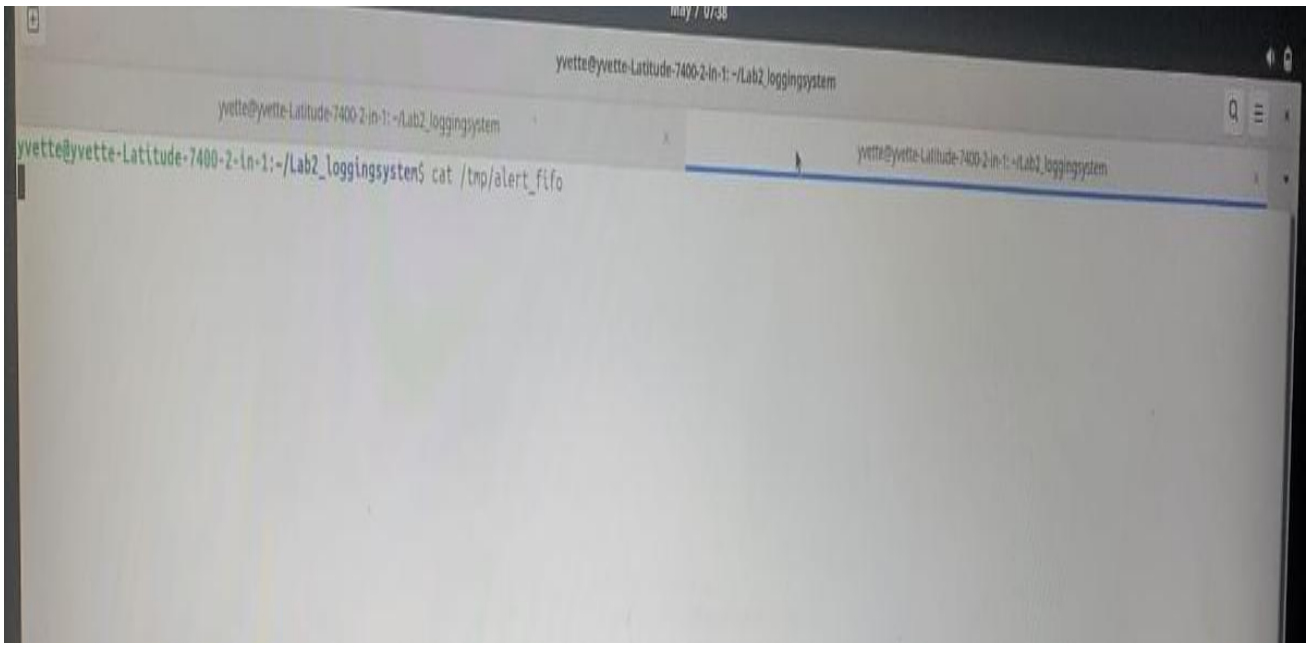
Message Format

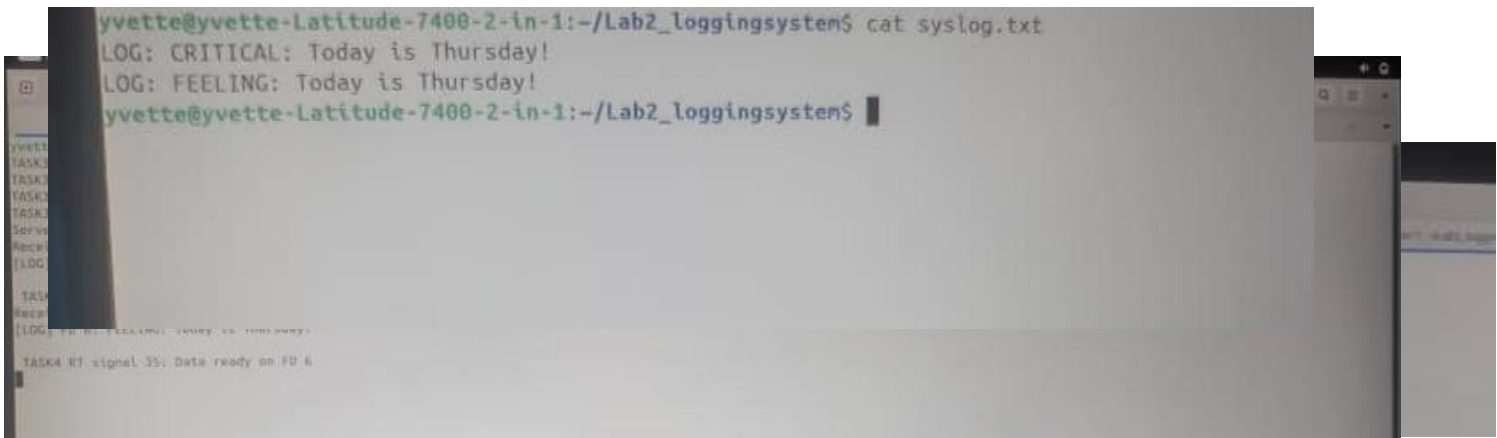
Messages follow the structure:

LEVEL|Message Content\n
Example:

INFO|System initialized successfully

Closing the FIFO generates an EOF condition which the server detects and handles appropriately.





Task 3: Dynamic Reconfiguration

To support runtime scalability, a control FIFO was implemented.

Control Commands

The server listens for commands such as:

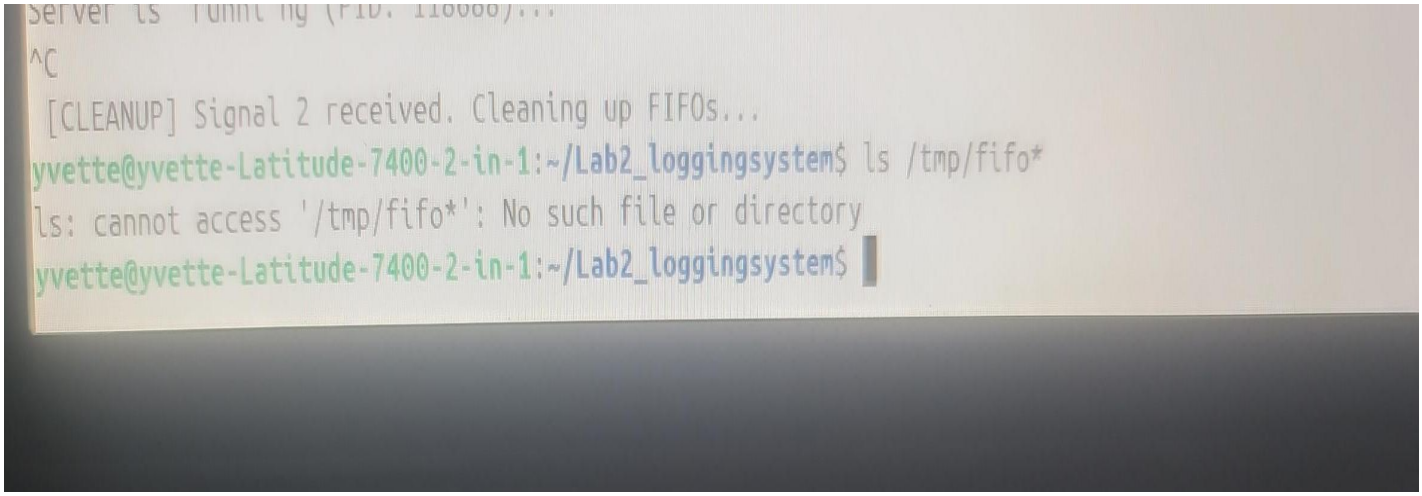
ADD /tmp/new_fifo

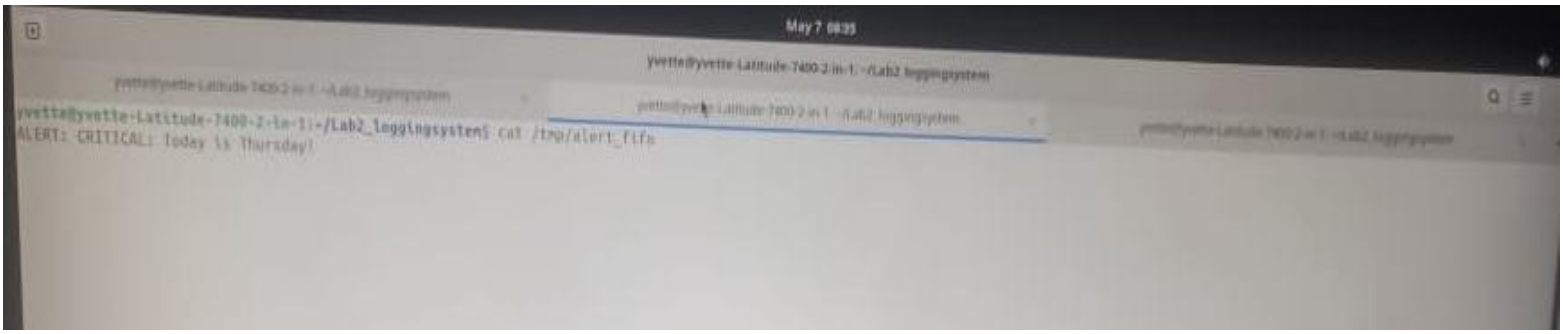
Dynamic Registration Process

Upon receiving an ADD command, the server dynamically:

- Creates the FIFO using mkfifo()
- Opens the FIFO in non-blocking mode
- Registers the descriptor using epoll_ctl(ADD)

This allows the system to scale without restarting the server.





Task 4: Real-Time Signals Exploration

The lab also investigated the use of real-time signals for asynchronous notifications.

Advantages

- Immediate notifications
- Reduced polling requirements
- Signal queuing support

Limitations

- Async-signal-safe restrictions
- Increased complexity
- Potential signal queue overflow
- Difficult synchronization

Although real-time signals are powerful, epoll proved more stable, efficient, and easier to manage for high-throughput logging systems.

8. Error Handling and Cleanup

8.1 Graceful Shutdown

A SIGINT handler was implemented to ensure proper cleanup during server termination.

The cleanup process includes:

- Closing all open descriptors
- Removing FIFOs using unlink()
- Releasing allocated resources

This prevents resource leaks and orphaned FIFO files.

8.2 Client Disconnection Handling

When a client disconnects:

- read() returns 0
- The server detects EOF
- The descriptor is removed from the epoll set
- Resources are released immediately

This improves resource efficiency and prevents descriptor exhaustion.

8.3 Handling EAGAIN

In non-blocking mode, read() may return EAGAIN when no additional data is available.

The server handles this condition correctly by:

- Stopping the current read cycle
- Waiting for the next epoll event

This behavior is essential in edge-triggered epoll systems.

Prelab Questions and Answers

Question 1

What is the difference between an unnamed pipe and a named pipe? Give a scenario where a named pipe is more suitable.

Answer

Unnamed pipes are created using pipe() and typically facilitate communication between related processes such as parent and child processes.

Named pipes (FIFOs) are created using mkfifo() and exist as files in the filesystem. They allow communication between unrelated processes.

A named pipe is more suitable in a centralized logging system where multiple independent applications send logs to a server.

Question 2

What does PIPE_BUF represent? Why is it important when sending messages through a pipe?

Answer

PIPE_BUF represents the maximum size of a write operation that the operating system guarantees to be atomic.

If data written to a pipe or FIFO is less than or equal to PIPE_BUF, the message will not interleave with writes from other processes.

This is important because it preserves message integrity in concurrent systems.

Question 3

Explain the difference between level-triggered and edge-triggered notifications in epoll. When would you choose one over the other?

Answer

Level-triggered mode continuously reports a descriptor as readable while unread data remains in the buffer.

Edge-triggered mode only reports events when new data arrives.

Level-triggered mode is easier to implement and suitable for simpler applications.

Edge-triggered mode is more efficient and reduces unnecessary wake-ups, making it ideal for high-performance servers.

Question 4

What system call is used to create a named pipe? What permissions are typically set?

Answer

The mkfifo() system call is used to create a named pipe.

Typical permissions include:

- 0666 for read/write access to all users
- 0644 for owner write access and public read access

Permissions depend on the required security level.

Question 5

Describe the steps to add a new FIFO to an existing epoll set.

Answer

The steps are:

1. Create the FIFO using mkfifo()
2. Open the FIFO using open()
3. Set non-blocking mode using O_NONBLOCK
4. Configure an epoll_event structure
5. Register the descriptor using epoll_ctl(ADD)

The server can then monitor the new FIFO without restarting.

Question 6

How do real-time signals differ from standard signals? What advantage does sigqueue() provide?

Answer

Real-time signals differ from standard signals because they:

- Are queued rather than merged
- Preserve delivery order
- Carry additional data

The sigqueue() function allows a process to attach an integer or pointer value to the signal, enabling richer asynchronous communication.

Question 7

If a client writes a message larger than PIPE_BUF, what might happen? How can you guarantee atomic delivery?

Answer

If a message exceeds PIPE_BUF, the operating system may split the write into multiple parts.

This can cause interleaving with data from other processes.

Atomic delivery can be guaranteed by:

- Keeping messages smaller than PIPE_BUF
- Using explicit framing or synchronization mechanisms

Question 8

Why is it necessary to handle EAGAIN when reading from a FIFO in non-blocking mode?

Answer

In non-blocking mode, read() returns EAGAIN when no data is currently available.

Handling EAGAIN is essential because:

- It prevents the application from blocking
- It indicates that all available data has been consumed
- It is required for proper operation in edge-triggered epoll mode

Advantages of the Implemented System

The developed system provides several advantages:

- High scalability
- Efficient event handling
- Low CPU overhead
- Reliable message integrity
- Runtime extensibility
- Non-blocking execution
- Efficient resource management

Limitations

Despite its advantages, the implementation also has some limitations:

- Edge-triggered programming complexity
- Dependence on Linux-specific APIs
- Potential signal queue overflow with real-time signals
- FIFO throughput limitations under extreme workloads

Future improvements could include:

- Thread pools
- Log persistence to databases

- Network socket integration
- Encryption and authentication
- Remote logging support

Conclusion

This laboratory exercise successfully demonstrated the implementation of a scalable logging server using Linux IPC and advanced event-driven programming techniques.

By combining:

- Named Pipes (FIFOs)
- Non-blocking I/O
- epoll multiplexing
- Edge-triggered event handling
- Dynamic runtime configuration
- POSIX atomic write guarantees

The system achieved:

- Efficient concurrency management
- Reliable message delivery
- Strong scalability
- Improved resource utilization

The exploration of real-time signals further highlighted the strengths of epoll as a robust and efficient event notification mechanism for high-performance Linux applications.

Overall, the project provided practical experience with advanced systems programming concepts and demonstrated how modern Linux facilities can be used to build efficient, scalable, and reliable server architectures.